AOSP 进阶丛书 · 第十一册 · ANDROID 13 (API 33)

存储与加密密钥全链

FBE 双域与 FsCrypt 逐段、密钥链五层（Gatekeeper→Synthetic Password→文件密钥）、keystore2 与 KeyMint 硬件仓库、direct boot 时间线、vold 卷管理与信任链四道墙收束。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

目 录

前言：数据的一生：加密、密钥与信任 2
第一章 FBE 双域：/data 的两把钥匙 2
第二章 密钥链：从手指到文件密钥 3
第三章 keystore2：Rust 重写的密钥仓库 5
第四章 direct boot：开机到解锁的时间线 6
第五章 vold：卷与存储的全职管家 6
第六章 信任链收束：四道墙拼图 7
第七章 设计决策九问 8
附录 A 观测实操 9
附录 B 快问快答（16 条） 9
附录 C 自检题 10
第八章 系统级钥匙：FsCrypt 初始化链全走读 10
第九章 解锁时刻：从输入 PIN 到 CE 密钥进内核 14
第十章 keystore2 走读：SELinux 域、super key 与锁定状态 19
第十一章 Direct Boot 机制：从 finishUserUnlocked 到 BOOT_COMPLETED 24
教科书习题集（分级） 28
术语表 31
排障手册：存储与加密问题定位 32
存储与加密核心速查表 35

本册速览

FBE 双域：DE(开机即解)与 CE(解锁后解)——FsCrypt.cpp 逐段
密钥链五层：PIN→Gatekeeper→Synthetic Password→文件密钥
keystore2(Rust)与 KeyMint(TEE/StrongBox)·attestation 证书链
direct boot 时间线与 vold 卷管理(FUSE/Checkpoint)
信任链四道墙收束：BootROM/AVB/dm-verity/fscrypt+KeyMint

前言：数据的一生：加密、密钥与信任

前十册讲"代码怎么跑"，本册讲"**数据怎么被保护**"：/data 上每个文件属于哪个加密域（FBE）、保护它的密钥从哪来到哪去（密钥链/KeyMint）、App 的私钥住在哪（keystore2）、重启后到解锁前的世界什么样（direct boot）、这一切与 AVB/dm-verity 如何拼成完整信任链。

源码基线不变（本树 Android 13）。主角目录：`system/vold`（FsCrypt/KeyStorage/Keymaster/VolumeManager）、`system/security/keystore2`（**Rust** 新世界）、`hardware/interfaces/keymint`（AIDL HAL）、`frameworks/base/services/core/.../locksettings`（LockSettingsService:202）。

第一章 FBE 双域：/data 的两把钥匙

设计洞察 · 学习目标

1. 说出 FBE 相比 FDE 的粒度差异与 direct boot 的物理基础；2. 读通 FsCrypt.cpp:842-873 的 DE 域目录建立与策略绑定三步；3. 区分"目录 Unix 权限"与"fscrypt 内容加密"两层保护；4. 画出 /data/user_de/N 与 /data/user/N 与 DE/CE 密钥的对应关系



图 1 FBE 双域：DE（开机即解）与 CE（解锁后解）的目录/住客/密钥来源，FsCrypt.cpp:842-873 逐段

1.1 逐段注释（真实源码）

cpp system/vold/FsCrypt.cpp:842-873（节选+讲解注释）

```
// 讲解注释①：BuildDataUserDePath(volume_uuid, user_id)（:842）
// ——DE 域目录寻址：/data/user_de/<id>。开机即挂（无需口令）。
auto user_de_path = android::vold::BuildDataUserDePath(volume_uuid, user_id);
// 讲解注释②：prepare_dir(:858)：建目录并落权限（0771 system:system）
// ——目录本身的可见性由 Unix 权限管，内容保密由 fscrypt 管。
if (!prepare_dir(user_de_path, 0771, AID_SYSTEM, AID_SYSTEM)) return false;
// 讲解注释③：EnsurePolicy(de_policy, user_de_path)（:873）
// ——给目录绑 fscrypt 加密策略：此后该目录下每个新文件
// 由内核按策略密钥独立加密（每文件 HKDF 派生子密钥）。
if (!EnsurePolicy(de_policy, user_de_path)) return false;
// CE 侧同构（user/0 目录），但装钥要等解锁（第六章 direct boot）。
```

先修要求与学习路线

先修：已读第一册（架构总览）。

读完本书你将能够：

- 解释存储与加密的核心机制与设计决策
- 追踪关键调用链并标注每次跨进程跳
- 使用 dumpsys/日志定位本领域问题
- 回答书7 对应章节的面试题

设计洞察 · 逐段精读：fscrypt 与 FDE 的世代差

FDE（全盘加密，EncryptInplace.cpp 的活化石）整盘一把钥匙、解锁前全系统不可用；FBE（文件级）按目录一域一钥——**重启后闹钟照响（DE）而微信看不到（CE）**成为可能。这就是 Android 7.1 以来 direct boot 的物理基础。

第二章 密钥链：从手指到文件密钥

设计洞察 · 学习目标

1. 默画密钥链五层（凭据/GateKeeper/SP/DE-CE Key/文件密钥）并标注每层的实现文件；2. 解释改 PIN 不重加密的间接层原理（SP 与 KEK 双份包装）；3. 说明 StrongAuth 重启后强制输 PIN 的规则本源；4. 说清换机/恢复出厂时各密钥的去向



图 2 密钥链五层：屏幕凭据→Gatekeeper→Synthetic Password→DE/CE Key→文件密钥；改 PIN 不重加密的秘密

2.1 链条逐层（含 LockSettingsService）

text 密钥链五层（路径+锚点）

```
① 屏幕凭据：PIN/图案/密码（+生物特征=另一把钥匙）
② Gatekeeper HAL（hardware/interfaces/gatekeeper/1.0，冻结维护）
——TEE 内校验+防暴力；框架只见"对/错/超时"签名
③ Synthetic Password（LockSettingsService.java:202 域）
——框架级"万能中间凭据"：CE key 由 SP 派生；
PIN/指纹都只是"解锁 SP 的钥匙"→ 改 PIN 不重加密全盘
④ DE/CE Key（vold KeyStorage.cpp；FsCrypt.cpp:195 KeyAuthentication）
——DE key 由硬件绑定 KMK 包装（无需口令）；CE key 由 SP 派生
⑤ 文件密钥：内核 fscrypt 按目录密钥 HKDF 派生（每文件独立）
```

> **设计洞察 · 面试核心：改密码为什么不用重加密**
>
> CE key 不随 PIN 变——变的是"SP 的包装"（KEK 双份：Gatekeeper 签名密钥+Weaver）。同理注册新指纹=给 SP 加一把钥匙，/data 毫无察觉。**间接层（SP）是密码学工程里的'稳定器'**。

2.3 密钥的落点与备份（数据侧最后一问）

"换机/恢复出厂密钥怎么办"三答案：**CE/DE 密钥永不迁移**（换机搬家的是密文+云端重下，不是密钥）；**keystore2 密钥不可导出**（KeyMint 硬件绑定——所以银行要在新机重新绑定）；**synthetic password 不参与云备份**（PIN 只在本机）。云备份（书6 § 三）备份的密文用云端密钥重加密——**两条密钥链（本机链/云链）互不相交**是"云备份可用性与本机安全"并存的设计底座。

2.2 防暴力与 StrongAuth

Gatekeeper 错误 N 次→StrongAuth 锁定（LockSettingsStrongAuth）：**重启后必须输 PIN（生物禁用）**的规则本源；Weaver（部分设备）是防离线爆破的硬件计数器。整个设计 TEE 边界清晰：**PIN 本体永不离开框架的加密包装**。

第三章 keystore2：Rust 重写的密钥仓库

> **设计洞察 · 学习目标**
>
> 1. 描述 keystore2 三层结构（App API/keystore2/KeyMint HAL）与 service.rs:54 的按安全级路由；2. 解释"句柄而非密钥"如何保证私钥不可导出；3. 列出授权三层（grant/用法/用户认证）并与权限三层对照；4. 用 dumpsys keystore2 观测条目与审计



图 3 keystore2 三层：App API→keystore2（service.rs:54 按安全级路由）→KeyMint HAL（TEE/StrongBox）

text system/security/keystore2（Rust）三层结构

```
App: android.security.keystore2.KeyGenParameterSpec
→ generateKey（App 只有句柄，无私钥本体）
keystore2: service.rs:54 i_sec_level_by_uuid（按 KeyMint 实例路由）
database.rs（条目+授权+审计） · async_task（队列） · attestation（证书链）
KeyMint HAL（hardware/interfaces/keymint/aidl）：
TEE 或 StrongBox（独立安全芯片）实现；
密钥永不出硬件；操作（签名/解密）进硬件执行
```

三条设计点：**句柄而非密钥**（App 侧导不出私钥——银行 App 的安全底气）；**授权三层**（条目 grant/用法 KeyGenParameterSpec/用户认证 setUserAuthenticationRequired——与书6 权限三层

同构的思想）；**attestation 证书链**（密钥生成时硬件签名证明"这密钥在这设备的安全硬件里"——Play Integrity 的信任根）。观测 `dumpsys keystore2`。

第四章 direct boot：开机到解锁的时间线

> **设计洞察 · 学习目标**
> 1. 画出 t0-t4 五点时间线并对应 vold/AMS 的源码动作；2. 说明 directBootAware 组件在 DE 期可用的条件与目录来源；3. 区分 ACTION_LOCKED_BOOT_COMPLETED 与 ACTION_BOOT_COMPLETED 的接收者差异；4. 识别"解锁前空数据"类 bug 的成因与修法



图 4 Direct Boot 时间线：t0 开机→t1 DE 解（vold 无口令装钥）→t2 锁屏→t3 输对凭据→t4 CE 解（USER_UNLOCKED 广播）

App 适配两件事：`directBootAware=true` 的组件在 DE 期即跑（只给闹钟/电话这类最小集）；常规 App 监听 ACTION_BOOT_COMPLETED 与 ACTION_USER_UNLOCKED **两个**广播分别处理。CE 未解时访问其数据是"挂起"不是崩溃。经典 bug：开机后通知点击打开 App 却是空数据=没处理解锁前空态。

第五章 vold：卷与存储的全职管家

设计洞察 · 学习目标

1. 描述 vold 作为 /data 唯一写者的职责边界（FBE 装钥/四类卷/FUSE/Checkpoint）；2. 追踪 VolumeManager::start 的启动链；3. 解释 FUSE 代理与 scoped storage 的物理执行关系；4. 用 vdc/sm 命令族观测卷状态



图 5 vold 四类卷生命周期（VolumeManager:173/:442）、FUSE 与 scoped storage、OTA 数据面 Checkpoint

vold（main.cpp:65；VolumeManager::start:173）是 /data 相关一切的唯一写者：FBE 装钥（前两章）、四类卷（private/public/adoptable/stub）、**FUSE 代理**——11 起 /storage 经 MediaProvider 进程审计改写（书6 scoped storage 的物理执行；13 的 passthrough 直通是性能回归）。**Checkpoint.cpp** 是 OTA 的数据面保险：与 A/B 的系统面（书2 § 十一）成对。客户端 `vdc/sm` 命令族。

第六章 信任链收束：四道墙拼图

设计洞察 · 学习目标

1. 列出四道信任墙各自的防护对象与源码落点；2. 推演解锁 bootloader 后四道墙各剩几道、attestation 如何标注；3. 把本册 fscrypt/KeyMint 与书2 AVB、书6 dm-verity 串联成完整链



图 6 信任链全景：BootROM→AVB（书2）→dm-verity（书6）→fscrypt/KeyMint（本册），各防什么与解锁后剩什么

四层职责不重叠：**防刷入（BootROM 熔丝）、防启动链篡改（AVB）、防运行时改系统（dm-verity）、防数据提取与密钥泄露（fscrypt+KeyMint）**。解锁 bootloader 后 AVB 橙警、verity 可禁，但 fscrypt 照常、KeyMint 照常（TEE 不看锁）——attestation 会标注解锁态，Play Integrity 据此识别。

第七章 设计决策九问

设计洞察 · 学习目标

1. 对九问各给出"问题/方案/代价"三段式回答；2. 至少追问三个"如果不用 X 会怎样"的反事实；3. 能把九问中的取舍迁移到自己的架构评审中

- **为什么 FBE 取代 FDE？** 粒度：按域解密让"重启后部分可用"成立；FDE 全有或全无，体验死锁。
- **为什么引入 Synthetic Password？** 换凭据不重加密（性能不可能每次全盘）；且支持"多把钥匙开同一扇门"（PIN+指纹）。
- **为什么 DE key 不需要口令？** 开机即用的服务（闹钟）无凭据可验——DE 的威胁模型本来就是"物理窃取后系统级保护"，KMK 硬件绑定足够。
- **为什么 keystore2 用 Rust 重写？** 并发安全（密钥操作的竞态代价=安全事故）；旧 Java keystore1 的授权模型僵硬（不能按时段/按用途细粒度）。
- **为什么 KeyMint 分 TEE/StrongBox 两级？** 成本与安全分级：StrongBox 独立芯片抗物理攻击更贵，高价值密钥（支付）才配。

- **为什么 attestation 是证书链不是单证书？** 设备根→中间→密钥三级可吊销可轮换（与 Web PKI 同构）。
- **为什么 FUSE 而不是内核原生 scoped storage？** 审计逻辑高频迭代（Mainline 可升级）；内核逻辑只能跟整机 OTA。
- **为什么 OTA 数据面要 Checkpoint？** A/B 只保系统面；应用数据被半 OTA 弄坏同样灾难——数据面快照补齐"全有或全无"。
- **为什么 Gatekeeper 冻结在 HIDL 1.0？** 接口极简（verify/enroll）无演进需求；迁移成本>收益（冻结也是架构决策）。

附录 A 观测实操

目标	命令/工具	看点
加密状态	dumpsys storageencryption	FBE 模式/密钥状态
vold 客户端	vdc cryptfs / vdc volume	shell 侧主力
keystore	dumpsys keystore2	条目/授权/审计
解锁状态	getprop sys.user.0.ce_available	CE 解锁事实
direct boot	dumpsys user	用户解锁状态
磁盘	sm list-disks / sm list-volumes	adoptable/storage
锁屏策略	cmd lock_settings get-disabled	StrongAuth 状态

附录 B 快问快答（16 条）

- FBE 与 FDE 一句话区别？——按目录分钥（域级）vs 全盘一钥。
- DE/CE 目录分别是什么？——/data/user_de/N 与 /data/user/N。
- 改 PIN 后 CE key 变吗？——不变（SP 间接层）。
- 指纹解锁的密码学含义？——SP 的另一把钥匙。
- Gatekeeper 在哪跑？——TEE/独立核（HAL 1.0 冻结）。
- keystore2 什么语言？——Rust（service.rs:54 路由 KeyMint）。
- StrongBox 与 TEE 区别？——独立安全芯片（更高抗物理，更贵）。
- attestation 给谁用？——远程证明（Play Integrity/银行）。
- 解锁前微信数据可见吗？——不可见（CE 未解，访问挂起）。
- USER_UNLOCKED 与 BOOT_COMPLETED 区别？——CE 解锁 vs 系统就绪（前者更晚）。
- FUSE 守门人是谁？——MediaProvider 进程（13 passthrough 优化）。

- adoptable 存储是什么？——SD 卡并入加密池变内部存储。
- OTA 的数据面保险？——vold Checkpoint（与 A/B 成对）。
- 解锁 bootloader 后 fscrypt？——仍生效（密钥随用户数据）。
- 四道信任墙？——BootROM/AVB/dm-verity/fscrypt+KeyMint。
- 本册之后？——书12 媒体（MediaDrm 用 KeyMint 分支）。

附录 C 自检题

- 默画密钥链五层与改 PIN 时的哪层变化。
- FsCrypt 三个锚点（:842/:858/:873）各做什么？
- direct boot 时间线五点；App 双广播适配。
- keystore2 三层与"导不出私钥"的机制保证。
- 解锁 bootloader 后四道墙各剩几道？

第八章 系统级钥匙：FsCrypt 初始化链全走读

> **设计洞察 · 学习目标**
> 1. 说出 FBE 初始化两步（enablefilecrypto/init_user0）的触发点与失败语义；2. 逐行读通 fscrypt_initialize_systemwide_keys 与 fscrypt_init_user0；3. 区分 device_key 与 per_boot_key 的落盘策略差异；4. 解释 v1 描述符与 v2 标识符的生成方与安全差异

链路位置：本章发生在"init 挂载 /data 之后、zygote 起来之前"——比第一章（概念）、第五章（vold 启动）更早，比第九章（用户 CE 钥匙）更早。完整调用链全部已核实：`init.rc:676 installkey /data` → **init 内建 do_installkey（system/core/init/builtins.cpp:1221-1229）** → `vdc enablefilecrypto`（`system/vold/vdc.cpp:115`） → `VoldNativeService::fbeEnable`（`VoldNativeService.cpp:554-559`） → `fscrypt_initialize_systemwide_keys`（`FsCrypt.cpp:435`）；随后 post-fs-data 尾部 `init.rc:976 init_user0` → `do_init_user0`（`builtins.cpp:1231-1233`） → `fscrypt_init_user0`（`FsCrypt.cpp:475`）。注意两步都经 `ExecVdcRebootOnFailure`：失败即触发重启——FBE 初始化是"失败关闭（fail-closed）"设计，宁可重启重试也不带病前进。

8.1 fscrypt_initialize_systemwide_keys：两把系统级钥匙

cpp system/vold/FsCrypt.cpp:435-470（节选+讲解注释）

```
bool fscrypt_initialize_systemwide_keys() {
LOG(INFO) << "fscrypt_initialize_systemwide_keys";
// 讲解注释①：从 ro.crypto.volume.options 读出内核策略参数：
// 内容/文件名加密算法、策略版本（v1/v2）、是否用硬件包装密钥。
EncryptionOptions options;
if (!get_data_file_encryption_options(&options)) return false;
// 讲解注释②：device_key 是"设备级 DE 主钥"。kEmptyAuthentication
```

```
// （KeyStorage.cpp:50，空 secret）表示无任何口令参与——开机即可
// 取回。首次开机走生成并落盘，之后开机走 retrieveKey。
KeyBuffer device_key;
if (!retrieveOrGenerateKey(device_key_path, device_key_temp, kEmptyAuthentication,
makeGen(options), &device_key))
return false;
// 讲解注释③：把 device_key 装进 /data 文件系统的 fscrypt keyring
// （ioctl FS_IOC_ADD_ENCRYPTION_KEY），内核回填 key_raw_ref。
EncryptionPolicy device_policy;
if (!install_storage_key(DATA_MNT_POINT, options, device_key, &device_policy)) return false;
// 讲解注释④：把策略参数与密钥引用写成"事实文件"持久化——路径
// 常量在 libfscrypt（system/extras/libfscrypt/include/fscrypt/
// fscrypt.h:24-27）：/data/unencrypted/mode 与 /unencrypted/ref。
std::string options_string;
if (!OptionsToString(device_policy.options, &options_string)) {
LOG(ERROR) << "Unable to serialize options";
return false;
}
std::string options_filename = std::string(DATA_MNT_POINT) + fscrypt_key_mode;
if (!android::vold::writeStringToFile(options_string, options_filename)) return false;
std::string ref_filename = std::string(DATA_MNT_POINT) + fscrypt_key_ref;
if (!android::vold::writeStringToFile(device_policy.key_raw_ref, ref_filename)) return false;
LOG(INFO) << "Wrote system DE key reference to:" << ref_filename;
// 讲解注释⑤：per_boot_key 每次开机重新生成、绝不落盘——它保护的
// 目录（引用写进 /data/unencrypted/per_boot_ref）重启后内容必然
// 不可恢复，实现"重启即清"语义，且不依赖硬件特性。
KeyBuffer per_boot_key;
if (!generateStorageKey(makeGen(options), &per_boot_key)) return false;
EncryptionPolicy per_boot_policy;
if (!install_storage_key(DATA_MNT_POINT, options, per_boot_key, &per_boot_policy)) return
false;
std::string per_boot_ref_filename = std::string("/data") + fscrypt_key_per_boot_ref;
if (!android::vold::writeStringToFile(per_boot_policy.key_raw_ref, per_boot_ref_filename))
return false;
LOG(INFO) << "Wrote per boot key reference to:" << per_boot_ref_filename;
return true;
}
```

设计洞察 · 逐段精读：一把落盘、一把不落

device_key 与 per_boot_key 同为"开机即装"的系统级钥匙，差别全在生命周期：device_key 经 KeyStorage 容器（第九章的五件套）持久化在 /data/unencrypted/key，跨重启稳定；per_boot_key 只把"引用"写进 per_boot_ref，密钥本体随内存消失——用它绑策略的目录天然获得"重启自毁"。这是用纯软件实现"易失存储域"的标准手法：**引用可持久，密钥不可持久**。

8.2 fscrypt_init_user0：user 0 的出生证明

cpp system/vold/FsCrypt.cpp:475-511（节选+讲解注释）

```
bool fscrypt_init_user0() {
LOG(DEBUG) << "fscrypt_init_user0";
// 讲解注释①：fscrypt_is_native 判定真 FBE；emulated 模式（
// persist.sys.emulate_fbe，FsCrypt.cpp:111-113）只是 chmod 模拟。
if (fscrypt_is_native()) {
```

```
// 讲解注释②：钥匙仓三连目录：/data/misc/vold/user_keys（:90）
// 及其 ce/、de/ 子目录，0700 root:root——App 永不可见。
if (!prepare_dir(user_key_dir, 0700, AID_ROOT, AID_ROOT)) return false;
if (!prepare_dir(user_key_dir + "/ce", 0700, AID_ROOT, AID_ROOT)) return false;
if (!prepare_dir(user_key_dir + "/de", 0700, AID_ROOT, AID_ROOT)) return false;
// 讲解注释③：user 0 的 DE 钥匙不存在则现场生成：首次开机的
// create_and_install_user_keys(:330) 一口气生成 DE/CE 两把。
if (!android::vold::pathExists(get_de_key_path(0))) {
if (!create_and_install_user_keys(0, false)) return false;
}
// 讲解注释④：load_all_de_keys()：把所有用户的 DE 钥匙装进内核
// keyring。DE 不需要凭据，开机早期即可全部就位——这就是 DE
// 域"开机即解"的实现点（源码 TODO 注明未来改为只装 DE_0）。
if (!load_all_de_keys()) return false;
}
// 讲解注释⑤：本函数只准备 DE 存储（STORAGE_FLAG_DE）！CE 目录
// 被刻意推迟——源码注释明说 CE keys "entangled with user
// credentials"（CE 钥匙缠绕着用户凭据，见第九章）。
if (!fscrypt_prepare_user_storage("", 0, 0, android::os::IVold::STORAGE_FLAG_DE)) {
LOG(ERROR) << "Failed to prepare user 0 storage";
return false;
}
// 讲解注释⑥：非 native 且非 emulated 的"刚离开模拟模式"设备：
// 用空 secret（"!"）把 user 0 目录拉回已知好状态。
if (!fscrypt_is_native() && !fscrypt_is_emulated()) {
fscrypt_unlock_user_key(0, 0, "!");
}
// 讲解注释⑦：userspace reboot 场景：umount /data 会丢掉 fs keyring
// 里的 CE 钥匙——这里尝试重装（弹药是 8.3 的 provisioning key）。
if (fscrypt_is_native() && android::vold::isFsKeyringSupported()) {
if (!try_reload_ce_keys()) return false;
}
// 讲解注释⑧：置位完成标志，框架侧据此判断 user0 初始化成功。
fscrypt_init_user0_done = true;
return true;
}
```

设计洞察 · 逐段精读：为什么 init_user0 不敢碰 CE

DE 钥匙的取回不需要任何用户输入，CE 钥匙的取回（设了锁屏时）必须先验证凭据。若在开机早期就给 CE 目录绑策略，要么绕过凭据（安全崩塌）要么让 zygote 等待输入（体验崩塌）。Android 的选择是**把 CE 目录的建立与策略绑定整体推迟到解锁时刻**（fscrypt_prepare_user_storage 的 CE 分支，FsCrypt.cpp:877-927 由解锁后的框架调用触发）——时间线上它正好落在第十一章 finishUserUnlocked 之前。

8.3 installKey：密钥描述符与策略安装

cpp system/vold/KeyUtil.cpp:268-318（节选+讲解注释）

```
bool installKey(const std::string& mountpoint, const EncryptionOptions& options,
const KeyBuffer& key, EncryptionPolicy* policy) {
// 讲解注释①：ioctl 参数连同裸密钥放进自动清零缓冲——vold 的内存
// 纪律：密钥材料用 KeyBuffer 携带，析构即擦除。
```

```
KeyBuffer arg_buf(sizeof(struct fscrypt_add_key_arg) + key.size(), 0);
struct fscrypt_add_key_arg* arg = (struct fscrypt_add_key_arg*)arg_buf.data();
// 讲解注释②：v1 与 v2 的"密钥名字"生成方不同：v1 用户态算描述符，
// v2 内核算标识符（用户给的值被忽略）。
switch (options.version) {
case 1:
// A key for a v1 policy is specified by an arbitrary 8-byte
// "descriptor" ... double SHA-512 of the key itself.
policy->key_raw_ref = generateKeyRef((const uint8_t*)key.data(), key.size());
if (!isFsKeyringSupported()) {
return installKeyLegacy(key, policy->key_raw_ref);
}
if (!buildKeySpecifier(&arg->key_spec, *policy)) return false;
break;
case 2:
// v2: 16-byte "identifier", a cryptographic hash of the key
// which the kernel computes and returns.
arg->key_spec.type = FSCRYPT_KEY_SPEC_TYPE_IDENTIFIER;
break;
default:
LOG(ERROR) << "Invalid encryption policy version: " << options.version;
return false;
}
// 讲解注释③：裸密钥填进 arg->raw，一次 ioctl 装进文件系统 keyring
//  （installFsKeyringKey 里 open /data 再 FS_IOC_ADD_ENCRYPTION_KEY，
// KeyUtil.cpp:250-266；硬件包装密钥在这里打 HW_WRAPPED 标志）。
arg->raw_size = key.size();
memcpy(arg->raw, key.data(), key.size());
if (!installFsKeyringKey(mountpoint, options, arg)) return false;
// 讲解注释④：v2 的标识符由内核现场计算并回填——同一把钥匙算出的
// 标识符恒定，可跨重启核对"策略有没有被偷换"。
if (arg->key_spec.type == FSCRYPT_KEY_SPEC_TYPE_IDENTIFIER) {
policy->key_raw_ref =
std::string((char*)arg->key_spec.u.identifier, FSCRYPT_KEY_IDENTIFIER_SIZE);
}
std::string ref = keyrefstring(policy->key_raw_ref);
LOG(DEBUG) << "Installed fscrypt key with ref " << ref << " to " << mountpoint;
// 讲解注释⑤：再把钥匙以 fscrypt-provisioning 类型挂进会话 keyring
//  （KeyUtil.cpp:190-211）：userspace reboot 后 try_reload_ce_keys
// 就从这里取回，免去再走一遍 KeyStorage 解密。
if (!installProvisioningKey(key, ref, arg->key_spec)) return false;
return true;
}
```

维度	v1 策略	v2 策略
密钥引用	8 字节描述符：用户态对密钥做 double SHA-512 取前 8 字节（KeyUtil.cpp:109-125）	16 字节标识符：内核计算并回填（KeyUtil.cpp:290-297）
撤销安全性	内核无法追踪哪些文件在用，移除不安全	内核记录文件使用计数，FS_IOC_REMOVE_ENCRYPTION_KEY 可安全清除
适用面	老内核（4.9 前）兼容	Android 11 起标配，本树默认

设计洞察 · 设计洞察：引用即名字，名字即信任

fscrypt 不存"密钥表"，文件 inode 里只放 8/16 字节的密钥引用；密钥本体住在内核 keyring。v2 让内核自己算引用，杜绝了用户态伪造"名字不同、密钥相同"的歧义——与 keystore2 用 uuid 路由 KeyMint 实例（第十章）同构：**名字由可信方生成，是整个寻址体系安全性的根**。

本章三个函数是所有后续故事的底座：第九章的 CE 解锁最终落到同一个 installKey；第十章 keystore2 的钥匙包装与 KeyStorage 容器（FsCrypt.cpp:87-88 的 /data/unencrypted/key 即 device_key 容器）互为表里；第十一章的 DE 期可用性完全依赖 8.2 的 load_all_de_keys。沿第四册的 init 视角看，installkey 与 init_user0 两个内建命令是 init.rc 少数"懂加密"的指令——init 只负责调度，密码学全在 vold。

第九章 解锁时刻：从输入 PIN 到 CE 密钥进内核

设计洞察 · 学习目标

1. 串起解锁一瞬间的 11 步链路并标注每步的文件与行号；2. 解释 KeyStorage 容器的两种包装模式（空 auth=KeyMint 包装、非空 auth=软件 AES-GCM）何时启用；3. 用"SP 不变则 fbe 派生密钥不变"解释改 PIN 时 vold 侧零动作；4. 说出 fscrypt_rewrap_user_key 在设锁/清锁时 retrieve 与 store 的 auth 配对

链路位置：第二章给出了五层概念图，本章把它展开为一次真实解锁的完整代码路径。前半（①-⑤）在 frameworks 的 locksettings 进程，后半（⑥-⑪）跨 binder 到 system_server 的 AMS/StorageManagerService、再到 vold 与 keystore2。第十一章将说明这之后 AMS 侧还差哪几步才算"解锁完成"。

text CE 钥匙链全链：解锁一瞬间的 11 步（全部锚点已核实）

```
① Keyguard 收集凭据 →
LockSettingsService.spBasedDoVerifyCredential（LockSettingsService.java:2886 起，入口 :
2286 checkCredential）
② SPM.computePasswordToken：scrypt(凭据, salt, N=1<<11, r=1<<3, p=1,
32B)（SyntheticPasswordManager.java
:1500-1504）
③ Weaver 设备：weaverVerify(slot, weaverKey)（SPM.java:1080）；否则 GateKeeper
verifyChallenge（SPM.java:1091
，TEE 内比对）
④ applicationId 重组 = pwdToken +
personalisedHash(secdiscardable)（transformUnderSecdiscardable SPM.ja
va:1385-1392）
⑤ unwrapSyntheticPasswordBlob：keystore2 auth-bound 密钥解出 256bit SP（SPM.java:1143；blob
创建 :1028-1048）
⑥ authToken.deriveDiskEncryptionKey() = SP800Derive(SP, "fbe-key")（SPM.java:213-215；V3 KDF
:195-203）
⑦ LSS.onCredentialVerified：unlockKeystore +
unlockUser(secret)（LockSettingsService.java:2975-3001，se
cret 即刻擦除 :2991）
⑧ AMS.unlockUser → UserController.unlockUserCleared →
StorageManager.unlockUserKey（UserController.ja
va:1751）
```

```
⑨ vold fscrypt_unlock_user_key：authentication_from_hex → 幂等检查（FsCrypt.cpp:752-764）
⑩ read_and_install_user_ce_key → KeyStorage.retrieveKey：同一套 appId 重组后解出 CE
钥匙（FsCrypt.cpp:296-308、Ke
yStorage.cpp:629-659）
⑪ installKey：CE 钥匙进内核 fs keyring，随后 EnsurePolicy 绑 CE
目录（KeyUtil.cpp:268-318，第八章 8.3）→ AMS 轮询到钥匙可用，推
进 finishUserUnlocked（第十一章）
```

9.1 SP 的创建与解包（frameworks 侧）

java frameworks/base/services/core/java/com/android/server/locksettings/SyntheticPasswordManager.java:791-849（节选+讲解注释）

```
// 讲解注释①：设锁时调用（改 PIN 也走这里）：生成新 handle，把"当前
// AuthenticationToken"重新包一层凭据。SP 本体从头到尾不变。
public long createPasswordBasedSyntheticPassword(IGateKeeperService gatekeeper,
LockscreenCredential credential, AuthenticationToken authToken, int userId) {
long handle = generateHandle();
PasswordData pwd = PasswordData.create(credential.getType());
// 讲解注释②：pwdToken = scrypt(凭据, salt)，32 字节。scrypt 参数存
// 在 pwd 里（N=2^11、r=2^3、p=2^1）：算得慢、耗内存——慢是为了
// 让离线爆破一张 PIN 表的代价大到不划算。
byte[] pwdToken = computePasswordToken(credential, pwd);
if (isWeaverAvailable()) {
// 讲解注释③：Weaver 路线：token 派生成 weaverKey 写入硬件槽位。
// 此后"验证凭据"是硬件内原子计数器比对：错 N 次硬件强制延时，
// 刷机/换系统也赖不掉（计数器在 RPMB 类存储里）。
int weaverSlot = getNextAvailableWeaverSlot();
Slog.i(TAG, "Weaver enroll password to slot " + weaverSlot + " for user " + userId);
byte[] weaverSecret = weaverEnroll(weaverSlot, passwordTokenToWeaverKey(pwdToken), null);
if (weaverSecret == null) {
throw new IllegalStateException("Fail to enroll user password under weaver " + userId);
}
saveWeaverSlot(weaverSlot, handle, userId);
// Weaver 模式没有 GateKeeper 句柄，SID 也置为无效（:814-815）。
pwd.passwordHandle = null;
sid = GateKeeper.INVALID_SECURE_USER_ID;
// 讲解注释④：两阶段解包的第一阶段产物：applicationId = pwdToken
// 拼 Weaver 吐回的 secret——硬件先解出 weaverSecret，才拼得出
// 解 SP blob 的钥匙（解锁侧对称流程见 SPM.java:1072-1086）。
applicationId = transformUnderWeaverSecret(pwdToken, weaverSecret);
} else {
// 讲解注释⑤：GateKeeper 路线：token 再做一次个性化哈希
//  （PERSONALIZATION_USER_GK_AUTH，:1506-1508）送 TEE enroll，
// 得到含 SID 的 passwordHandle——比对只能在 TEE 内发生。
response = gatekeeper.enroll(fakeUid(userId), null, null,
passwordTokenToGkInput(pwdToken));
pwd.passwordHandle = response.getPayload();
sid = sidFromPasswordHandle(pwd.passwordHandle);
// 讲解注释⑥：applicationId = pwdToken + hash(16KB 随机
// secdiscardable)。secdiscardable 一旦销毁，blob 永久打不开
// ——这是 SP"安全删除"的根，与 vold KeyStorage 同思路。
applicationId = transformUnderSecdiscardable(pwdToken,
createSecdiscardable(handle, userId));
}
```

```
// 讲解注释⑦：三件套落盘：pwd 元数据（scrypt 参数/句柄）、加密的
// metrics（密码强度，用 SP 派生密钥加密）、以及 SP blob 本体。
saveState(PASSWORD_DATA_NAME, pwd.toBytes(), handle, userId);
savePasswordMetrics(credential, authToken, handle, userId);
createSyntheticPasswordBlob(handle, SYNTHETIC_PASSWORD_PASSWORD_BASED, authToken,
applicationId, sid, userId);
return handle;
}
```

设计洞察 · 设计洞察：SP blob 的"双因子"结构

解开 SP blob 需要 applicationId = pwdToken + hash(secdiscardable)。pwdToken 只能由"正确的凭据 +TEE/GateKeeper 或 Weaver 判定通过"得到，secdiscardable 是磁盘上的随机件——删除它等于销毁 SP。凭据是"你知的"，secdiscardable 是"盘上的"：两者缺一，暴力离线爆破只能对着密文干瞪眼。这与 vold KeyStorage 的 secdiscardable 用法（下节）完全同构，是 Android 密码学工程的家族基因。

9.2 一把 SP 派生全家桶

java frameworks/base/services/core/java/com/android/server/locksettings/SyntheticPasswordManager.java:195-228（节选+讲解注释）

```
// 讲解注释①：SP 的"一钥多职"全靠这条派生函数：同一 SP，不同
// personalization 串派生不同用途的钥匙。V3 起用 NIST SP800-108
// KDF（SP800Derive.java）带上下文与固定串；V1/V2 退化为
// personalisedHash（SHA512）。
private byte[] derivePassword(byte[] personalization) {
if (mVersion == SYNTHETIC_PASSWORD_VERSION_V3) {
return (new SP800Derive(mSyntheticPassword))
.withContext(personalization, PERSONALISATION_CONTEXT);
} else {
return SyntheticPasswordCrypto.personalisedHash(personalization,
mSyntheticPassword);
}
}
// 讲解注释②：keystore2 口令（hex 串）：通知 keystore2 解锁该用户的
// auth-bound 超级密钥层（走 Authorization.onLockScreenEvent，
// LockSettingsService.java:1375-1378，第十章接棒）。
public byte[] deriveKeyStorePassword() {
return bytesToHex(derivePassword(PERSONALIZATION_KEY_STORE_PASSWORD));
}
// 讲解注释③：送 GateKeeper 的 SP 输入：刷新 SID 对应的 auth token。
public byte[] deriveGkPassword() {
return derivePassword(PERSONALIZATION_SP_GK_AUTH);
}
// 讲解注释④：本章主角：送 vold 解 CE 容器的 secret——personalization
// 串是 "fbe-key"（SPM.java:136）。改 PIN 不换 SP，所以它永不变：
// 这就是"改密码不重加密"在 vold 侧零动作的数学根源。
public byte[] deriveDiskEncryptionKey() {
return derivePassword(PERSONALIZATION_FBE_KEY);
}
// 讲解注释⑤：同一 SP 还派生 vendor AuthSecret（引导加载器共享口令）
// 与 metrics 加密钥——一次解锁，全家可用。
public byte[] deriveVendorAuthSecret() {
return derivePassword(PERSONALIZATION_AUTHSECRET_KEY);
```

```
}
// 讲解注释⑥：密码 metrics（长度/字符类别）的加密钥：系统要评估密码
// 强度却不必存明文——又一个"派生而非复用"的范例。
public byte[] deriveMetricsKey() {
return derivePassword(PERSONALIZATION_PASSWORD_METRICS);
}
```

9.3 LSS 的解锁总闸门与 vold 交界

java frameworks/base/services/core/java/com/android/server/locksettings/LockSettingsService.java:2975-3001 与 :2117-2125（讲解注释）

```
// 讲解注释①：凭据验证成功后的总闸门：authToken 里已带着解出的 SP。
private void onCredentialVerified(AuthenticationToken authToken, PasswordMetrics metrics,
int userId) {
if (metrics != null) {
synchronized (this) {
mUserPasswordMetrics.put(userId, metrics);
}
} else {
Slog.wtf(TAG, "Null metrics after credential verification");
}
// 讲解注释②：第一步：通知 keystore2 解锁该用户（deriveKeyStore-
// Password 即 SP800Derive(SP, "keystore-password") 的 hex）。
unlockKeystore(authToken.deriveKeyStorePassword(), userId);
// 讲解注释③：第二步：vold 解 CE。派生值用完立即内存擦除
//  （Arrays.fill）——跨进程传递一次就够，框架不留副本。
{
final byte[] secret = authToken.deriveDiskEncryptionKey();
unlockUser(userId, secret);
Arrays.fill(secret, (byte) 0);
}
// 讲解注释④：第三步：设备管理器注册的 escrow token 一并激活。
activateEscrowTokens(authToken, userId);
if (isProfileWithSeparatedLock(userId)) {
setDeviceUnlockedForUser(userId);
}
// 讲解注释⑤：StrongAuth 计时重置（第二章 2.2 的"重启后强制输
// PIN"由此计数回归正常）。
mStrongAuth.reportSuccessfulStrongAuthUnlock(userId);
onAuthTokenKnownForUser(userId, authToken);
}
// 讲解注释⑥：unlockUser(:1436) 走 AMS：mActivityManager.unlockUser(
// userId, null, secret, listener)——进入第十一章的 UserController
// 状态机。而下面这个直连 StorageManager 的助手用在"清除凭据"路径
//  （:3053-3060，注释原文：clear key from vold so ActivityManager
// can just unlock the user with empty secret during boot）。
private void unlockUserKey(int userId, byte[] secret) {
final UserInfo userInfo = mUserManager.getUserInfo(userId);
try {
mStorageManager.unlockUserKey(userId, userInfo.serialNumber, secret);
} catch (RemoteException e) {
throw new IllegalStateException("Failed to unlock user key " + userId, e);
}
}
```

cpp system/vold/FsCrypt.cpp:752-778（节选+讲解注释）

```
// 讲解注释①：vold 的解锁入口（IVold.aidl:95）。secret_hex 是框架送来
// 的 SP800Derive(SP, "fbe-key") 字节串的 hex 形式；serial 是用户
// serial 号——防"删用户后新建同号用户"错拿旧钥匙。
bool fscrypt_unlock_user_key(userid_t user_id, int serial, const std::string& secret_hex) {
LOG(DEBUG) << "fscrypt_unlock_user_key " << user_id << " serial=" << serial;
if (fscrypt_is_native()) {
// 讲解注释②：幂等保护：s_ce_policies 是"本进程已装 CE 钥匙"
// 的内存表（fscrypt_get_unlocked_users :743-749 也靠它）。
if (s_ce_policies.count(user_id) != 0) {
LOG(WARNING) << "Tried to unlock already-unlocked key for user " << user_id;
return true;
}
// 讲解注释③：hex→KeyAuthentication（:637-646）。secret 非空时
// KeyAuthentication::usesKeystore()（KeyStorage.h:34）为 false
// ——决定下节容器按"软件包装"路径解。
auto auth = authentication_from_hex(secret_hex);
if (!auth) return false;
// 讲解注释④：读 CE 钥匙并装进内核 keyring，成功后登记进
// s_ce_policies；此后 prepare_user_storage 才敢给 CE 目录绑策略。
if (!read_and_install_user_ce_key(user_id, *auth)) {
LOG(ERROR) << "Couldn't read key for " << user_id;
return false;
}
// 讲解注释④b：read_and_fixate（:194-209）按目录序逐个试版本，
// 成功者改名 current 并销毁其余版本——多版本是重包过程的
// 原子性保险（temp+rename，第九章 9.4）。
} else {
// 讲解注释⑤：emulated 模式：chmod 模拟四个 CE 目录的解锁。
if (!emulated_unlock(android::vold::BuildDataSystemCePath(user_id), 0771) ||
!emulated_unlock(android::vold::BuildDataMiscCePath("", user_id), 01771) ||
!emulated_unlock(android::vold::BuildDataMediaCePath("", user_id), 0770) ||
!emulated_unlock(android::vold::BuildDataUserCePath("", user_id), 0771)) {
LOG(ERROR) << "Failed to unlock user " << user_id;
return false;
}
}
return true;
}
```

设计洞察 · 逐段精读：KeyStorage 容器五件套与双包装

每个 CE 钥匙目录（/data/misc/vold/user_keys/ce/N/current）装五件：version（"1"，retrieveKey 先核对）、secdiscardable（16KB 随机，哈希进 appId）、stretching（包装模式记录）、keymaster_key_blob（仅空 auth 模式）、encrypted_key。storeKey（KeyStorage.cpp:575-608）按 auth 分两路：**auth 为空 → KeyMint 包装**（generateKeyStorageKey :135-143 在 TEE 生成 wrapping key，AES-256-GCM 三段式 :376-400）；**auth 非空 → 软件包装**（appId = secdiscardable_hash + secret，密钥=SHA512("Android key wrapping key generation SHA512" + appId) :468-472）。无锁屏用户走前者（TEE 才能解），设了锁屏走后者（secret 本身已由 TEE 判定的凭据保护）。

9.4 设锁与清锁：rewrap 协议

设锁时（LSS.setUserKeyProtection :2074 → addUserKeyAuth :2127 → IVold.addUserKeyAuth，IVold.aidl:90）vold 执行 `fscrypt_add_user_key_auth`（FsCrypt.cpp:713-719）→ `fscrypt_rewrap_user_key`（:687-711）：**用 kEmptyAuthentication 取回（KeyMint 路径）、用新 secret 存回（软件路径）**；清锁时 `fscrypt_clear_user_key_auth`（:721-727）反向配对。改 PIN 则两者都不发生——SP 不变，`deriveDiskEncryptionKey()` 不变，vold 只在 SP blob 层（第九章 9.1 的 createPasswordBasedSyntheticPassword 与 :3017-3078 的 setLockCredentialWithAuthTokenLocked）换包装。`:699` 的兜底分支（用空 auth 再试一次）对应"swipe 与 none 互切时框架会带 secret 调 clear"的历史行为（:696-698 TODO 注释原文）。

> **设计洞察 · 设计洞察：两把 CE 钥匙模式的威胁模型**
> 无锁屏：CE 容器 KeyMint 包装，TEE 是唯一钥匙扣——纯软件攻击者复制的镜像解不开。设锁屏：容器由"secdiscardable+SP 派生 secret"的软件密钥包装，而 secret 的获取已由 GateKeeper/Weaver 在 TEE/硬件内把关。两种模式殊途同归：**TEE/硬件总在链路里，区别只在于它把守的是容器本身还是容器的钥匙**。

交叉引用：本章 ⑩ 步的 retrieveKey 细节、storeKeyAtomically 的"temp+rename+fsync"原子落盘（KeyStorage.cpp:610-627）与 destroyKey 的 secdiscard（:687-713）承第八章 8.3 的 installKey；解锁完成后的 AMS 侧推进见第十一章；keystore2 如何用 deriveKeyStorePassword 解锁超级密钥见第十章。

第十章 keystore2 走读：SELinux 域、super key 与锁定状态

> **设计洞察 · 学习目标**
> 1. 读通 KeystoreService 的双 HashMap 路由与 TEE 强制/StrongBox 可选的装配逻辑；2. 说清 Domain::APP 与 Domain::SELINUX 双轨命名空间及 keystore2_key_contexts 的 7 个系统域；3. 用 keystore2 三态 UserState（Uninitialized/LskfLocked/LskfUnlocked）解释设备锁定对 auth 密钥可用性的影响；4. 说明 vold 为何只用 TEE 不用 StrongBox

链路位置：第三章给了 keystore2 三层鸟瞰，本章进入 Rust 源码。keystore2 是第九章 ⑤ 步（解 SP blob）与 ⑦ 步 unlockKeystore 的接收方，也是 vold 包装 CE/DE 容器（第八/九章 KeyStorage）的 HAL 中介。三条线在"设备锁定状态"汇合：同一个 TEE，vold 看不见锁、keystore2 却看得见。

10.1 服务装配：uuid 与安全级的双向映射

text system/security/keystore2/src/service.rs:52-96（节选+讲解注释）

```
// 讲解注释①：KeystoreService 只有两个 HashMap：每个 KeyMint 实例
//  （TEE/StrongBox）一个 uuid；密钥条目在数据库里记住自己出生在
// 哪个 uuid——访问时按 uuid 找回正确的 HAL 代理（:106-113）。
```

```
pub struct KeystoreService {
i_sec_level_by_uuid: HashMap<Uuid, Strong<dyn IKeystoreSecurityLevel>>,
uuid_by_sec_level: HashMap<SecurityLevel, Uuid>,
}
impl KeystoreService {
pub fn new_native_binder(
id_rotation_state: IdRotationState,
) -> Result<Strong<dyn IKeystoreService>> {
let mut result: Self = Default::default();
// 讲解注释②：TEE 安全层是强制的：构造失败整个服务起不来。
let (dev, uuid) = KeystoreSecurityLevel::new_native_binder(
SecurityLevel::TRUSTED_ENVIRONMENT,
id_rotation_state.clone(),
)
.context(concat!(
"In KeystoreService::new_native_binder: ",
"Trying to construct mandatory security level TEE."
))?;
result.i_sec_level_by_uuid.insert(uuid, dev);
result.uuid_by_sec_level.insert(SecurityLevel::TRUSTED_ENVIRONMENT, uuid);
// 讲解注释③：StrongBox 可选：没有独立安全芯片的设备忽略错误
// ——服务照常运行，STRONGBOX 请求在 get_security_level(:115-129)
// 返回 HARDWARE_TYPE_UNAVAILABLE。
if let Ok((dev, uuid)) =
KeystoreSecurityLevel::new_native_binder(SecurityLevel::STRONGBOX, id_rotation_state)
{
result.i_sec_level_by_uuid.insert(uuid, dev);
result.uuid_by_sec_level.insert(SecurityLevel::STRONGBOX, uuid);
}
let uuid_by_sec_level = result.uuid_by_sec_level.clone();
// 讲解注释④：keystore1 遗留数据懒迁移：首次访问旧条目时才搬
//  （get_key_entry 里的 with_try_import，:139）。
LEGACY_IMPORTER
.set_init(move || {
(create_thread_local_db(), uuid_by_sec_level, LEGACY_BLOB_LOADER.clone())
})
.context(
"In KeystoreService::new_native_binder: Trying to initialize the legacy migrator.",
)?;
Ok(BnKeystoreService::new_binder(
result,
BinderFeatures { set_requesting_sid: true, ..BinderFeatures::default() },
))
}
}
```

get_key_entry（service.rs:131-180）是 App 每次 getKeyEntry 的必经之路：先取调用方 uid 对应的超级密钥缓存（:134-135，`get_per_boot_key_by_user_id`——user 未解锁时为 None，auth 密钥后续解不开）、再经 LEGACY_IMPORTER 试迁移、最后 `load_key_entry` 做域/权限检查并回填证书链与授权集。服务注册名 `android.system.keystore2.IKeystoreService/default`（keystore2_main.rs:33；vold 侧同名常量 Keystore.cpp:28-29 双向核对）。

10.2 SELinux 域划分：系统组件的密钥隔离

namespace	SELinux 标签	使用方
0	su_key	su 调试域（native 测试）
1	shell_key	shell 域（native 测试）
100	vold_key	vold：裸 KeyMint blob（Keystore.cpp:38 注明与该文件保持同步）
101	odsign_key	odsign：设备内签名守护
102	wifi_key	Wi-Fi 子系统（替代 keystore1 的 WIFI_UID 命名空间）
103	locksettings_key	LockSettingsService
120	resume_on_reboot_key	Reboot on Reboot（服务器侧托管解密）

上表来自 `system/sepolicy/private/keystore2_key_contexts:8-27`（格式：namespace 标签）。keystore2 在权限检查时把 Domain::SELINUX 的 namespace 经 `lookup_keystore2_key_context`（permission.rs:46）翻成标签，再与调用方 `getcon()`（permission.rs:357）比对。

> **设计洞察 · 设计洞察：App 域与 SELinux 域双轨**
>
> keystore2 密钥寻址有两条轨：Domain::APP 按 Linux uid 划分命名空间（App 互不可见，普通应用永远在这条轨）；Domain::SELINUX 按 SELinux 上下文划分（系统组件用——它们可能同 uid（都是 system），能隔离它们的只有 SELinux）。第六章"四道墙"里"KeyMint 照常"的微观保障正是这两层：**uid 隔离给 App，标签隔离给系统，TEE 隔离给所有人**。

10.3 super key：keystore2 自己的用户钥匙

text system/security/keystore2/src/super_key.rs:543-615（节选+讲解注释）

```
// 讲解注释①：输对锁屏凭据（LSKF=锁屏知识因子）后，keystore2 用它
// 解出该用户的超级密钥——keystore2 侧的"CE 钥匙"，管理它加密的
// 所有 auth-bound App 密钥。
pub fn check_and_unlock_super_key(
&mut self, db: &mut KeystoreDB, legacy_importer: &LegacyImporter,
user_id: UserId, pw: &Password,
) -> Result<UserState> {
let alias = &USER_SUPER_KEY;
// 讲解注释②：先看旧格式（legacy keystore1）有没有超级钥匙（迁移
// 窗口），再从数据库载入"被密码加密的 super key blob"。
let result = legacy_importer
.with_try_import_super_key(user_id, pw, || db.load_super_key(alias, user_id))
.context("In check_and_unlock_super_key. Failed to load super key")?;
match result {
Some((_, entry)) => {
// 讲解注释③：用密码解出 super key 并放进内存缓存：从首次
```

```
// 解锁常驻到重启（AfterFirstUnlock 语义，:83-86）。
let super_key = self
.populate_cache_from_super_key_blob(user_id, alias.algorithm, entry, pw)
.context("In check_and_unlock_super_key.")?;
Ok(UserState::LskfUnlocked(super_key))
}
// 讲解注释④：数据库里没有超级钥匙 = 该用户从未设锁屏：返回
// Uninitialized（创建首把密钥时再按需初始化）。
None => Ok(UserState::Uninitialized),
}
}
// 讲解注释⑤：首次设锁路径：生成新 AES-256 super key，立刻用密码
//  （源码注释：derive an AES256 key from the password and re-encrypt
// the super key before insert；+Weaver 若可用）加密入库，然后进缓存。
pub fn check_and_initialize_super_key(
&mut self, db: &mut KeystoreDB, legacy_importer: &LegacyImporter,
user_id: UserId, pw: Option<&Password>,
) -> Result<UserState> {
let super_key_exists_in_db = self
.super_key_exists_in_db_for_user(db, legacy_importer, user_id)
.context("In check_and_initialize_super_key. Failed to check if super key exists.")?;
if super_key_exists_in_db {
Ok(UserState::LskfLocked)
} else if let Some(pw) = pw {
let super_key = generate_aes256_key()
.context("In check_and_initialize_super_key: Failed to generate AES 256 key.")?;
let (encrypted_super_key, blob_metadata) = Self::encrypt_with_password(&super_key, pw)
.context("In check_and_initialize_super_key.")?;
let key_entry = db
.store_super_key(user_id, &USER_SUPER_KEY, &encrypted_super_key,
&blob_metadata, &KeyMetaData::new())
.context("In check_and_initialize_super_key. Failed to store super key.")?;
let super_key = self
.populate_cache_from_super_key_blob(user_id, USER_SUPER_KEY.algorithm, key_entry, pw)
.context("In check_and_initialize_super_key.")?;
Ok(UserState::LskfUnlocked(super_key))
} else {
Ok(UserState::Uninitialized)
}
}
```

每个用户有**三层**超级密钥（UserSuperKeys，super_key.rs:242-258；类型定义 :83-100）：

层	别名	常驻期	典型使用方
per_boot	USER_SUPER_KEY	首次解锁到重启	普通 auth 密钥（AfterFirstUnlock）
screen_lock_bound	USER_SCREEN_LOCK_BOUND_KEY	每次输 LSKF 到锁屏	锁屏敏感密钥（支付/凭据确认）
screen_lock_bound_private	USER_SCREEN_LOCK_BOUND_P521_KEY	同上（ECDH 私钥）	锁定态下仍需"可加密不可解密"的场景

keystore2 UserState	含义	auth 密钥操作的结局
Uninitialized	该用户从未设锁屏	创建密钥时按需初始化 super key
LskfLocked	设了锁屏但本次开机未解锁	super_encrypt 返回 LOCKED（super_key.rs:721）
LskfUnlocked	本次开机输过一次 LSKF	per_boot 层可用；screen-lock-bound 层另判

设计洞察 · 设计洞察：keystore2 与 vold 的"锁"不同频

vold 的 CE 只有"解锁/未解锁"两态且一次性；keystore2 多出 screen-lock-bound 一档——锁屏（非重启）即可收回。App 侧 setUserAuthenticationRequired 的密钥在锁屏后被拒、解锁后自动恢复，靠的就是这一层的内存纪律。同一台设备上 vold 觉得"解锁过"，keystore2 可能仍"锁着"：两者各守各的频率，互不代替。

10.4 vold 与 keystore2 的交界

cpp system/vold/Keystore.cpp:106-151（节选+讲解注释）

```
// 讲解注释①：vold 侧的 keystore2 客户端构造：等服务上线再取代理。
Keystore::Keystore() {
::ndk::SpAIBinder binder(AServiceManager_waitForService(keystore2_service_name));
auto keystore2Service = ks2::IKeystoreService::fromBinder(binder);
if (!keystore2Service) {
LOG(ERROR) << "Vold unable to connect to keystore2.";
return;
}
// 讲解注释②：只取 TEE 级。源码注释给出理由（:115-123）：有 TEE
// 就有 Weaver，CE 已被 Weaver 强化，StrongBox 无增量收益——
// 安全设计要"对"而不是"堆"。
auto rc = keystore2Service->getSecurityLevel(km::SecurityLevel::TRUSTED_ENVIRONMENT,
&securityLevel);
if (logKeystore2ExceptionIfPresent(rc, "getSecurityLevel"))
LOG(ERROR) << "Vold unable to get security level from keystore2.";
}
bool Keystore::generateKey(const km::AuthorizationSet& inParams, std::string* key) {
// 讲解注释③：Domain::BLOB + VOLD_NAMESPACE（100，即 vold_key）：
// blob 直接回给调用方落盘，不进 keystore2 数据库——vold 要把
// blob 存进自己的 KeyStorage 五件套（第九章）。
ks2::KeyDescriptor in_key = {
.domain = ks2::Domain::BLOB,
.alias = std::nullopt,
.nspace = VOLD_NAMESPACE,
.blob = std::nullopt,
};
ks2::KeyMetadata keyMetadata;
auto rc = securityLevel->generateKey(in_key, std::nullopt, inParams.vector_data(), 0, {},
&keyMetadata);
if (logKeystore2ExceptionIfPresent(rc, "generateKey")) return false;
if (keyMetadata.key.blob == std::nullopt) {
LOG(ERROR) << "keystore2 generated key blob was null";
return false;
```

```
}
// 讲解注释④：拿到 blob 立刻清零源缓冲（zeroize_vector）——C++ 侧
// 与 Java 侧（KeyBuffer/Arrays.fill）同一条内存纪律。
if (key) *key = std::string(keyMetadata.key.blob->begin(), keyMetadata.key.blob->end());
zeroize_vector(keyMetadata.key.blob.value());
return true;
}
```

第四种用途在 exportKey（Keystore.cpp:153-181）：对带 TAG_STORAGE_KEY 的密钥调用 convertStorageKeyToEphemeral——硬件生成的 inline-encryption key 以"临时密钥"形式吐出，供带硬件包装密钥特性的设备（generateWrappedStorageKey，KeyStorage.cpp:145-155）给 fscrypt 做真正的每文件硬件加密。:169-175 注释明确：vold 忽略 upgradedBlob——STORAGE_KEY 的版本演进交给 vold 自己的包装层处理。

交叉引用：第三章（三层鸟瞰与本节代码对应）、第九章 ⑤/⑦ 步（unlockKeystore 与 unwrap SP blob 都终结在 keystore2）、第六章（解锁 bootloader 不影响本章任何逻辑——TEE 不看 bootloader 状态，attestation 另有标记）、书12（MediaDrm 的密钥分支走本章的 security_level 代理）。

第十一章 Direct Boot 机制：从 finishUserUnlocked 到 BOOT_COMPLETED

设计洞察 · 学习目标

1. 把第四章的 t0-t4 时间线落到 UserController 的状态机与广播序列；2. 说明 DE 存储的建立时机（prepareUserStorage）与 directBootAware 组件的数据来源；3. 区分 ACTION_LOCKED_BOOT_COMPLETED、ACTION_USER_UNLOCKED、ACTION_BOOT_COMPLETED 三个广播的接收者与时刻；4. 用 StorageManagerService 的 ce_available 门禁解释"解锁前 mkdirs 抛异常"的现象

链路位置：第九章 ⑧ 步把 secret 交给了 AMS，本章接着讲 AMS 侧：unlockUserCleared 的状态机、finishUserUnlocked 的完成时刻、以及两个 BOOT 广播的分界。第八章 8.2 已交代 DE 侧前提（load_all_de_keys 开机即装），本册的故事在此收束。

text Direct Boot 状态机（frameworks 侧，锚点全部核实）

```
t0 开机：init 挂 /data（第八章两步）→ zygote → system_server
t1 AMS.startUser → UserController.unlockUserCleared（UserController.java:1743）
└ StorageManager.isUserKeyUnlocked=false → IStorageManager.unlockUserKey（:1751）
（vold 以空 secret 取 DE 钥匙装内核——无需凭据，第九章⑨步的空 auth 版本）
t2 状态 STATE_RUNNING_LOCKED：directBootAware 组件可跑（只给 DE 存储）
└ 广播 ACTION_LOCKED_BOOT_COMPLETED（sendLockedBootCompletedBroadcast :615-625）
t3 用户输对凭据 → 第九章全链 → vold 装好 CE 钥匙
t4 StorageManager.isUserKeyUnlocked=true → finishUserUnlocked（:687）
└ 状态 STATE_RUNNING_UNLOCKING → STATE_RUNNING_UNLOCKED（:697）
└ 启动 encryption-unaware 组件（:707-709）、广播 ACTION_USER_UNLOCKED（:713）
t5 finishUserUnlockedCompleted（:766）→ 广播 ACTION_BOOT_COMPLETED（:830-845）
```

11.1 unlockUserCleared：解锁总入口

java frameworks/base/services/core/java/com/android/server/am/UserController.java:1743-1780（节选+讲解注释）

```
// 讲解注释①：解锁一个用户的总入口：开机启动用户（maybeUnlockUser
// :1731-1733）与输 PIN 后的 LSS.unlockUser（LockSettingsService.java:
// 1461，经 AMS）都汇到这里。
private boolean unlockUserCleared(final @UserIdInt int userId, byte[] secret,
IProgressListener listener) {
UserState uss;
if (!StorageManager.isUserKeyUnlocked(userId)) {
final UserInfo userInfo = getUserInfo(userId);
final IStorageManager storageManager = mInjector.getStorageManager();
try {
// 讲解注释②：secret 可能为 null：开机自启场景走 DE 解锁——
// vold 侧 fscrypt_unlock_user_key 收到空 hex 时返回
// kEmptyAuthentication（FsCrypt.cpp:641-642），空 auth 取
// DE 钥匙无需凭据；输 PIN 场景带第九章的 fbe 派生 secret。
// We always want to unlock user storage, even user is not started yet
storageManager.unlockUserKey(userId, userInfo.serialNumber, secret);
} catch (RemoteException | RuntimeException e) {
Slogf.w(TAG, "Failed to unlock: " + e.getMessage());
}
}
// 讲解注释③：进度监听挂到 UserState：锁屏 UI 的"正在解锁"转圈
// 就订阅在这条 progress 上（LSS:1442-1458 传进来的 listener）。
synchronized (mLock) {
// Register the given listener to watch for unlock progress
uss = mStartedUsers.get(userId);
if (uss != null) {
uss.mUnlockProgress.addListener(listener);
}
}
// Bail if user isn't actually running
if (uss == null) {
notifyFinished(userId, listener);
return false;
}
// 讲解注释④：finishUserUnlocking 推进状态机到 UNLOCKING 并
// 触发 LOCKED_BOOT_COMPLETED；CE 真正就绪后 finishUserUnlocked
// 接棒（下节）。
final TimingsTraceAndSlog t = new TimingsTraceAndSlog();
t.traceBegin("finishUserUnlocking-" + userId);
final boolean finishUserUnlockingResult = finishUserUnlocking(uss);
t.traceEnd();
if (!finishUserUnlockingResult) {
notifyFinished(userId, listener);
return false;
}
// 讲解注释⑤：主用户解锁后立刻联动解锁其下托管 profile：遍历
// 已启动用户，凡是 parent 已解锁的 profile 递归 maybeUnlockUser
//  （:1789-1800；反向的"等父解锁"在 :596-613）。
```

（尾部省略的是 :1789-1800 的 profile 联动循环——遍历 mStartedUsers 并对 parent 已解锁的 profile 调 maybeUnlockUser。）

11.2 finishUserUnlocked：CE 就绪的时刻

java frameworks/base/services/core/java/com/android/server/am/UserController.java:687-763（节选+讲解注释）

```
private void finishUserUnlocked(final UserState uss) {
final int userId = uss.mHandle.getIdentifier();
EventLog.writeEvent(EventLogTags.UC_FINISH_USER_UNLOCKED, userId);
// 讲解注释①：闸门：CE 钥匙没装进内核就原路返回。本函数被多个
// 事件（用户启动/钥匙装好/父用户解锁）反复触发——谁最后一个
// 到，谁负责把状态机推过去。
// Only keep marching forward if user is actually unlocked
if (!StorageManager.isUserKeyUnlocked(userId)) return;
synchronized (mLock) {
// Bail if we ended up with a stale user
if (mStartedUsers.get(uss.mHandle.getIdentifier()) != uss) return;
// Do not proceed if unexpected state
if (!uss.setState(STATE_RUNNING_UNLOCKING, STATE_RUNNING_UNLOCKED)) {
return;
}
}
mInjector.getUserManagerInternal().setUserState(userId, uss.state);
uss.mUnlockProgress.finish();
// 讲解注释②：user 0：启动 persistent 应用里"不感知加密"的那批
//  （MATCH_DIRECT_BOOT_UNAWARE）——数据在 CE 里，此刻才可读。
if (userId == UserHandle.USER_SYSTEM) {
mInjector.startPersistentApps(PackageManager.MATCH_DIRECT_BOOT_UNAWARE);
}
// 讲解注释③：给完全不感知加密的内容提供者补装代理——DE 期它们
// 没跑，App 在 DE 期查询它们会被挂起，此刻补上。
mInjector.installEncryptionUnawareProviders(userId);
if (!mInjector.getUserManager().isPreCreated(userId)) {
// Dispatch unlocked to external apps
final Intent unlockedIntent = new Intent(Intent.ACTION_USER_UNLOCKED);
unlockedIntent.putExtra(Intent.EXTRA_USER_HANDLE, userId);
unlockedIntent.addFlags(
Intent.FLAG_RECEIVER_REGISTERED_ONLY | Intent.FLAG_RECEIVER_FOREGROUND);
mInjector.broadcastIntent(unlockedIntent, null, null, 0, null,
null, null, AppOpsManager.OP_NONE, null, false, false, MY_PID, SYSTEM_UID,
Binder.getCallingUid(), Binder.getCallingPid(), userId);
}
// 讲解注释④：托管 profile 解锁后向父用户发 PROFILE_ACCESSIBLE 与
// MANAGED_PROFILE_UNLOCKED（:722-747，此处省略）。
// 讲解注释⑤：OTA 后指纹变化才发 PRE_BOOT——BOOT_COMPLETED 被
// 刻意压到 PRE_BOOT 全部收尾之后（注释原文：purposefully block
// sending BOOT_COMPLETED until after all PRE_BOOT receivers are
// finished to avoid ANR'ing apps）。
final UserInfo info = getUserInfo(userId);
if (!Objects.equals(info.lastLoggedInFingerprint, PackagePartitions.FINGERPRINT)
|| SystemProperties.getBoolean("persist.pm.mock-upgrade", false)) {
final boolean quiet = info.isManagedProfile();
mInjector.sendPreBootBroadcast(userId, quiet,
() -> finishUserUnlockedCompleted(uss));
} else {
finishUserUnlockedCompleted(uss);
}
```

```
}
```

设计洞察 · 逐段精读：t4 为什么依赖轮询而非回调

vold 装好钥匙后并不会"回调"AMS——UserController 是在多次触发里反复检查 `StorageManager.isUserKeyUnlocked`（isUserKeyUnlocked 读的是 StorageManagerService 维护的用户钥匙状态，其底层事实来自 vold）。这种"幂等检查+状态机推进失败即返回"的模式（:691/:697）让多个并发事件源天然收敛，不需要精确的事件顺序——分布式状态机的低配解法，却非常稳健。

11.3 三个广播的分界

java frameworks/base/services/core/java/com/android/server/am/UserController.java:615-625（节选+讲解注释）

```
// 讲解注释①：t2 时刻的广播：只送达 manifest 里 directBootAware 的
// 接收者（系统按组件属性过滤，App 无需自行判断）。
private void sendLockedBootCompletedBroadcast(IIntentReceiver receiver, @UserIdInt int
userId) {
final Intent intent = new Intent(Intent.ACTION_LOCKED_BOOT_COMPLETED, null);
intent.putExtra(Intent.EXTRA_USER_HANDLE, userId);
intent.addFlags(Intent.FLAG_RECEIVER_NO_ABORT
| Intent.FLAG_RECEIVER_OFFLOAD
| Intent.FLAG_RECEIVER_INCLUDE_BACKGROUND);
// 讲解注释②：权限与普通 BOOT_COMPLETED 同守（RECEIVE_BOOT_
// COMPLETED）；临时 allowlist 保证开机早期广播可达性。
mInjector.broadcastIntent(intent, null, receiver, 0, null, null,
new String[]{android.Manifest.permission.RECEIVE_BOOT_COMPLETED},
AppOpsManager.OP_NONE,
getTemporaryAppAllowlistBroadcastOptions(REASON_LOCKED_BOOT_COMPLETED)
.toBundle(), true, ...);
}
```

（尾部参数省略同源码 :625。）BOOT_COMPLETED 在 finishUserUnlockedCompleted 里发出：`final Intent bootIntent = new Intent(Intent.ACTION_BOOT_COMPLETED, null);`（UserController.java:830-845，经 FgThread 保证与 widget 广播的顺序）。三个广播一表定分界：

广播	时刻	接收者	数据可见性
ACTION_LOCKED_BOOT_COMPLETED	t2（DE 就绪即发）	仅 directBootAware 组件	只有 DE
ACTION_USER_UNLOCKED	t4（CE 钥匙装好）	注册型接收者（REGISTERED_ONLY）	DE+CE
ACTION_BOOT_COMPLETED	t5（PRE_BOOT 收尾后）	全部组件	DE+CE

11.4 DE 存储从哪来：prepareUserStorage 与 App 门禁

DE 期目录不是广播时才建：StorageManagerService 在用户启动路径上调 `prepareUserStorage`（StorageManagerService.java:3375-3384 → `mVold.prepareUserStorage`，IVold.aidl:98）→ vold `fscrypt_prepare_user_storage`（FsCrypt.cpp:810-930）按 **STORAGE_FLAG_DE 分支**（:832-875）

建 /data/system_de、/data/misc_de、/data/vendor_de、/data/user_de 并用第八章装好的 DE 钥匙 EnsurePolicy；CE 分支（:877-927）在解锁后由框架再次调用时才执行。directBootAware 组件拿到的 /data/user_de 就是这一步的产物；它访问 CE 路径会被"挂起"（pending unlock）而非报错。

App 侧还有一个更早的门禁：`mkdirs`（StorageManagerService.java:3711-3726）在 `isUserKeyUnlocked` 为假时直接抛 `IllegalStateException`，并在 user 0 上检查属性 `sys.user.N.ce_available`（:3715）。

> **设计洞察 · 经典 bug 复盘：开机点击通知打开空数据页**
> 因果链：闹钟/消息是 directBootAware 组件在 DE 期发出 → 用户点击后启动的 Activity 属普通组件却立刻读 CE 数据 → CE 未解锁，查询挂起或返回空 → 页面空白。两种修法：数据侧，把"必须立刻可见"的字段冗余进 DE（directBootAware 的第二个 receiver/文件）；时序侧，Activity 检测 UserManager.isUserUnlocked，未解锁就跳"解锁后继续"引导页并注册 ACTION_USER_UNLOCKED。判据：**任何在 LOCKED_BOOT_COMPLETED 里启动的 UI 都必须假设 CE 不可用**。

交叉引用：第四章（时间线概念版）、第八章 8.2（DE 钥匙开机即装是 t1/t2 的前提）、第九章（t3 的完整密码学链路）、第五章（vold prepareUserStorage 是"卷与存储管家"职责的一部分）、附录 A（getprop/dumpsys 观测点）。

教科书习题集（分级）

第一级：简答题（检验概念理解）

- **简答 1**：FBE 与 FDE 的本质区别是什么？各带来怎样的解锁体验？

  参考答案要点：1. FDE 整盘一把钥匙，解锁前系统完全不可用；2. FBE 按域分钥（DE 开机即解、CE 解锁后解）；3. 体验差异=direct boot 是否可能：重启后闹钟响（DE）而 App 数据不可见（CE）。

- **简答 2**：DE 与 CE 目录分别是什么？各自的钥匙由谁保护？

  参考答案要点：1. /data/user_de/N 与 /data/user/N（第一章）；2. DE 钥匙以 kEmptyAuthentication 存放、开机由 load_all_de_keys 装载（FsCrypt.cpp:475-511）；3. CE 钥匙容器受"凭据派生 secret"或 KeyMint 包装，解锁时装载。

- **简答 3**：改 PIN 为什么不需要重加密？指出哪层变、哪层不变。

  参考答案要点：1. 变的是 SP 的包装（新 handle/scrypt 参数/GateKeeper 句柄，SPM.java:791-849）；2. SP 本体与 deriveDiskEncryptionKey() 派生值不变（SPM.java:213-215）；3. vold 侧 CE 容器与文件密钥完全零动作（第九章 9.4）。

- **简答 4**：device_key 与 per_boot_key 有何差异？

  参考答案要点：1. device_key 持久化于 /data/unencrypted/key，跨重启稳定；2. per_boot_key 每次开机重新生成、只留引用于 per_boot_ref（FsCrypt.cpp:461-468）；3. 后者实现"重启即清"语义。

- **简答 5**：Synthetic Password 是什么？它解决了什么问题？

  参考答案要点：1. 随机生成的 256bit 中间凭据 hash(P0+P1)（SPM.java:261-280）；2. PIN/指纹都只是"解锁 SP 的钥匙"；3. 支持多凭据开同一锁与改凭据不动密钥——间接层是稳定器。

- **简答 6**：KeyStorage 容器的两种包装模式分别何时启用？

  参考答案要点：1. auth 为空（无锁屏）→ KeyMint wrapping key 包装（KeyStorage.cpp:588-598）；2. auth 非空（设了锁屏）→ 软件 AES-256-GCM，密钥=SHA512(前缀+secdiscardable_hash+secret)（:468-472）；3. 设锁/清锁由 fscrypt_rewrap_user_key 在两模式间切换（FsCrypt.cpp:687-727）。

- **简答 7**：keystore2 的 Domain::APP 与 Domain::SELINUX 有何区别？

  参考答案要点：1. APP 按 Linux uid 划分命名空间，App 互不可见；2. SELINUX 按 SELinux 标签划分，服务同 uid 仍隔离（permission.rs:46/357）；3. 系统域映射见 keystore2_key_contexts（100=vold_key、103=locksettings_key 等）。

- **简答 8**：keystore2 的 UserState 三态是什么？对 auth 密钥各有何影响？

  参考答案要点：1. Uninitialized（未设锁屏，按需初始化 super key）；2. LskfLocked（本次开机未解锁，auth 操作返回 LOCKED，super_key.rs:721）；3. LskfUnlocked（per_boot 层常驻到重启，screen-lock-bound 层锁屏即收回）。

- **简答 9**：ACTION_LOCKED_BOOT_COMPLETED、ACTION_USER_UNLOCKED、ACTION_BOOT_COMPLETED 三者在时刻与接收者上如何分界？

  参考答案要点：1. LOCKED_BOOT_COMPLETED 在 DE 就绪（STATE_RUNNING_LOCKED，仅 directBootAware 组件，UserController.java:616）；2. USER_UNLOCKED 在 CE 钥匙装好（:713，注册型接收者）；3. BOOT_COMPLETED 在 PRE_BOOT 收尾后全员可达（:830）。

- **简答 10**：secdiscardable 在 vold KeyStorage 与 SyntheticPasswordManager 中各起什么作用？

  参考答案要点：1. vold：16KB 随机件，哈希进 appId，删除即容器永久不可解（KeyStorage.cpp:191-204）；2. SPM：SP blob 的第二因子（SPM.java:1385-1392）；3. 共同思想：凭据（你知的）+盘上随机件=双因子，且删除随机件=安全删除。

第二级：分析题（检验机制理解）

- **分析 1**：串出"输对 PIN 到 /data/user/0 可读"的完整 11 步链，标注每步文件与行号。

  参考答案要点：1. spBasedDoVerifyCredential→scrypt token→GK/Weaver 判定→applicationId 重组→解 SP blob→deriveDiskEncryptionKey→onCredentialVerified→AMS.unlockUser→vold unlock→retrieveKey→installKey 进内核（第九章链路表为标准答案）。

- **分析 2**：反事实推演：若 fscrypt_init_user0 直接绑定 CE 目录（跳过 STORAGE_FLAG 分离），会发生什么？

参考答案要点：1. 要么在无凭据时用 CE 钥匙=安全模型崩塌；要么 zygote 等待输入=开机死锁；2. 正确设计把 CE 目录建立推迟到解锁后（FsCrypt.cpp:488-490 注释本意）；3. 由此理解 direct boot 的工程代价。

- **分析 3**：改密码的完整协议：SP 层、vold 层、keystore2 层各变什么？

参考答案要点：1. SP 层：新 handle+新 GK enroll，SP 不变（LSS:3017-3044）；2. vold 层：零动作（派生 secret 不变）；3. keystore2 层：per_boot super key 由 LSKF 派生包装同步更新（super_key.rs encrypt_with_password）；4. 结论：改 PIN 的成本=一次 SP blob 重包+超级密钥重包，与数据量无关。

- **分析 4**：userspace reboot 后 CE 钥匙为何会丢？靠哪两个机制找回？

参考答案要点：1. umount /data 丢 fs keyring（FsCrypt.cpp:502-504 注释）；2. 机制一：installProvisioningKey 把钥匙挂在会话 keyring（KeyUtil.cpp:190-211）；3. 机制二：try_reload_ce_keys 从会话 keyring 重装（FsCrypt.cpp:505-507）。

- **分析 5**：OTA 升级 KeyMint 后 blob 升级为何必须延迟到 checkpoint 提交？

参考答案要点：1. blob 升级=旧 blob 作废；若 OTA 回滚则旧 blob 已删=密钥丢失；2. KeyStorage.cpp:365-372 检测 cp_needsCheckpoint 后只写 _upgraded 文件并入队（:238-269）；3. vold.checkpoint_committed=1 后 DeferredCommitKeys 才替换并删旧 key——密钥面与系统面同原子。

- **分析 6**：keystore2 的 screen-lock-bound 层与 LskfBound 层分别适配什么密钥？失败语义各是什么？

参考答案要点：1. LskfBound（per_boot）：首次解锁后常驻，适配"解锁一次即可"的普通密钥；2. screen-lock_bound：锁屏即收回内存，适配支付确认类；3. 锁定态操作返回 ResponseCode::LOCKED（super_key.rs:716-721）；4. P521 变体支持锁定态"只加密"。

- **分析 7**：复盘"开机点击通知打开空数据页"：给出完整因果链与两种修法。

参考答案要点：1. 因果：DE 期通知（directBootAware 组件）→ 点击启动普通 Activity → CE 未解锁访问挂起 → 空页面；2. 修法一：关键字段冗余到 DE；3. 修法二：isUserUnlocked 检测+引导页+监听 USER_UNLOCKED（第十一章 11.4 盒）。

第三级：设计题

- **设计 1**：为"换机迁移"设计密钥链方案：哪些密钥可迁移、哪些必须不可迁移、工作资料的 PIN 如何处理？给出与 D2D 传输的分工。

参考答案要点：1. CE/DE 密钥永不迁移（迁移密文重加密或云端重下）；2. keystore2 硬件绑定密钥在新机重新生成、attestation 区分；3. 工作资料 PIN 可随 D2D 迁移凭据本身（SP 层），但目标机 SP 必须重绑定目标机 GateKeeper/Weaver；4. 落点：受本册 2.3 与第九章协议约束。

- **设计 2**：为一个"重启后 direct boot 期即可用的快速回复"功能设计完整方案（数据、密钥、组件、广播四个清单）。

参考答案要点：1. 数据：消息头/模板入 DE 或 directBootAware ContentProvider；2. 密钥：发送凭据走 DE 可取路径，敏感正文等 CE；3. 组件：receiver+process 声明 directBootAware=true，Activity 不声明；4. 广播：LOCKED_BOOT_COMPLETED 起服务、USER_UNLOCKED 后合并 CE 历史记录（第十一章）。

- **设计 3**：若把 keystore2 数据库整体迁入独立分区（或 pVM 场景），重新评估其域划分、super key 存放与用户解锁状态传递机制。

参考答案要点：1. SELinux 标签需为新域扩展 keystore2_key_contexts；2. super key 的"密码加密入库"模式不依赖数据库位置，但早解锁信号（onLockScreenEvent）需跨 VM 认证通道；3. 锁定态与 vold CE 解锁态跨 VM 传递需防重放（对标 resume_on_reboot_key 的服务器托管思路）。

术语表

术语	解释
安全丢弃种子（secdiscardable）	与密钥同目录存放的 16KB 随机件，其哈希参与 appId 派生；删除它=容器永久不可解，是"安全删除"的实现根（KeyStorage.cpp:191-204、SPM.java:1385-1392）
超级密钥（super key）	keystore2 为每个设锁用户生成的 AES-256 钥匙，用它超级加密该用户的 auth-bound 密钥；被 LSKF 派生密钥包装后入库（super_key.rs:543-615）
合成密码（Synthetic Password，SP）	框架层随机生成的 256bit 中间凭据：CE 钥匙、keystore 口令、AuthSecret 全由它派生；PIN/指纹只是"解锁 SP 的钥匙"（SPM.java:179-228）
回滚抵抗（rollback resistance）	KeyMint 特性：密钥删除由硬件确认、不可恢复旧版本；vold 生成容器钥匙时优先申请、失败则降级（KeyStorage.cpp:120-133）
检查点（Checkpoint）	vold 的 /data 数据面快照：OTA 期间写时重定向，失败可整体回滚；与 A/B 系统面成对（Checkpoint.cpp、vdc checkpoint 命令族）
每次启动密钥（per-boot key）	每次开机重新生成、不落盘的 fscrypt 钥匙，引用写入 /data/unencrypted/per_boot_ref；实现"重启即清"（FsCrypt.cpp:461-468）
密钥槽位（Weaver slot）	Weaver 硬件里"key/value+计数器"槽：存凭据派生 key 与 secret，验证失败有硬件强制延时；slot 由 PasswordSlotManager 全局分配（SPM.java:766-775）
密钥环（keyring）	内核密钥容器：fscrypt 钥匙经 FS_IOC_ADD_ENCRYPTION_KEY 装入文件系统 keyring，另有会话 keyring 挂 fscrypt-provisioning 钥匙供 userspace reboot 重装（KeyUtil.cpp:155-211）

术语	解释
密钥描述符与标识符（key descriptor/identifier）	fscrypt v1 的 8 字节用户态描述符（double SHA-512）与 v2 的 16 字节内核计算标识符；文件 inode 只存引用（KeyUtil.cpp:109-125、290-297）
密钥证明（attestation）	KeyMint 生成的证书链：证明密钥存在于本设备安全硬件及其授权属性；解锁 bootloader 会留下可检测标记，Play Integrity 的信任根
凭据加密（CE，Credential Encrypted）	需用户凭据解锁的加密域：目录 /data/user/N、/data/system_ce 等，钥匙容器由 SP 派生 secret 或 KeyMint 包装（第九章）
强身份验证（StrongAuth）	锁定策略机制：重启/超时后要求完整凭据（禁生物），LockSettingsStrongAuth 计数，成功解锁后由 LSS:2998 复位
认证令牌（AuthenticationToken）	SPM 中持有 SP 及其派生值的运行时对象：deriveDiskEncryptionKey/deriveKeyStorePassword 等派生口即定义于此（SPM.java:195-228）
设备加密（DE，Device Encrypted）	开机即解的加密域：目录 /data/user_de/N 等，钥匙无凭据保护、开机由 load_all_de_keys 装载（FsCrypt.cpp:486）
锁屏知识因子（LSKF，Lock Screen Knowledge Factor）	PIN/图案/密码的统称；keystore2 与 LSS 都以"验证 LSKF"为解锁原语（super_key.rs 注释）
文件级加密（fscrypt）	内核按目录策略逐文件加密的机制：每文件密钥由目录钥匙经 KDF 派生；v1/v2 策略差异见第八章 8.3
信任链（trust chain）	BootROM→AVB→dm-verity→fscrypt/KeyMint 四道墙（第六章）；本册负责最后一道：数据提取防护与密钥不出硬件
硬件包装密钥（hw-wrapped key，TAG_STORAGE_KEY）	KeyMint 生成的 inline-encryption key：硬件内可转换出临时密钥供 fscrypt 直用（KeyStorage.cpp:145-166、Keystore.cpp:153-181）
直接启动（Direct Boot）	DE 解锁到 CE 解锁之间系统仍可服务的模式：物理基础是 FBE 双域，时间线见第四章与第十一章
直接启动感知（directBootAware）	组件属性：声明后可在 DE 期运行并访问 /data/user_de；LOCKED_BOOT_COMPLETED 只送达这类接收者（UserController.java:616）

排障手册：存储与加密问题定位

总法：`dumpsys lock_settings`/`dumpsys keystore2`/`dumpsys user` 定位状态 → `logcat` 按下述 tag 精读 → 按各章深入。以下五个场景覆盖本册 90% 的线上问题，所有 log 字样均摘自本树源码。

场景一：重启后 CE 域一直未解锁（应用数据空、访问挂起）

症状：开机后应用数据为空、`mkdirs` 抛 `Failed to prepare`（StorageManagerService.java:3719 门禁，user 0 还会检查属性 `sys.user.0.ce_available`:3715）。logcat 关键字样例：`FsCrypt: fscrypt_unlock_user_key 0 serial=0`（FsCrypt.cpp:753）；`KeyStorage: decryptWithKeystoreKey failed`（KeyStorage.cpp:649）；`SyntheticPasswordManager: gatekeeper verify failed`（SPM.java:1094）；`LockSettingsService: Auth token not available for user 0`（LSS:841）。

定位命令：`logcat -s FsCrypt KeyStorage SyntheticPasswordManager LockSettingsService`；`cmd lock_settings verify --old <PIN>`（LockSettingsShellCommand.java:48）；`dumpsys lock_settings`（看 SP handle 与 StrongAuth）；`vdc cryptfs init_user0`（DE 侧自检，vdc.cpp:117）。

根因：三类之一：凭据错误（GateKeeper 返回 RETRY 计时中，SPM.java:1123-1125）；keystore2/KeyMint 侧解 SP blob 失败（⑤ 步，常伴随 OTA 后 blob 未升级提交）；CE 钥匙容器损坏（`FsCrypt: Failed to find working ce key for user 0`，FsCrypt.cpp:207——所有版本都试尽）。

修复：凭据类等待 StrongAuth 冷却后重试；blob 类核对 `getprop vold.checkpoint_committed` 与 `dumpsys keystore2`（见场景四）；容器损坏属于"设计性不可恢复"——secdiscardable 与密文任一损坏即永久不可解（第九章），只能恢复出厂。诊断时先区分"设计性拒绝"与"真实损坏"：前者换了正确凭据能恢复。

场景二：输对 PIN 也被拒（跨设备/刷机后）

症状：logcat 样例：`SyntheticPasswordManager: Credential type mismatch: expected %d actual %d`（SPM.java:1062-1063）；Weaver 设备：`weaver read failed (INCORRECT_KEY), slot: N`（SPM.java:543）后跟 `VerifyCredentialResponse.fromTimeout`。

定位命令：`logcat -s SyntheticPasswordManager`；`cmd lock_settings get-disabled`（生物禁用/StrongAuth 状态，LockSettingsShellCommand.java:49）；`dumpsys user`（确认目标 user 是否存在历史快照）。

根因：a) PIN 实际属于另一用户/工作资料；b) 用户数据被整盘复制到新机：Weaver 计数器与 GK blob 均与本机硬件绑定，INCORRECT_KEY 是预期行为而非 bug；c) GK 句柄过期——verifyChallenge 返回 shouldReEnroll 后系统会自动 re-enroll（SPM.java:1101-1122），失败只发生在句柄已损坏时。

修复：a/b 类无解（凭据与硬件绑定是设计）；c 类让用户用最近一次设置的凭据重试触发 re-enroll。跨机复制数据导致"永远解不开"正是 FRP/防盗模型的正确行为。

场景三：keystore 操作失败（App 签名/解密抛异常）

症状：App 侧 `UserNotAuthenticatedException/KeyPermanentlyInvalidatedException`；logcat 样例：`keystore2: ... Error::Rc(ResponseCode::LOCKED)`（设备锁定时 super_encrypt 拒绝，super_key.rs:721）；`In get_key_entry: KeyMint instance for key not found`（service.rs:106-113）。

定位命令：`dumpsys keystore2`（条目/授权/审计）；`logcat -s keystore2`；确认解锁状态 `dumpsys user | grep -i unlocked`。

根因：a) 设备锁定+密钥 setUserAuthenticationRequired：LskfLocked 态被拒（设计，第十章 10.3 表）；b) 密钥属另一用户/SELinux 域越权（permission.rs:357-359）；c) OTA 后 KeyMint 版本升级尚未完成（auth 操作触发 upgrade，见场景四同款机制）。

修复：a) 解锁后重试或在解锁期内预取；b) 核对密钥生成时的 user/域；c) 完成一次解锁让 keystore2 完成 blob 升级与缓存填充。上线前用"锁定态调用 auth 密钥"作为标准用例。

场景四：OTA 后卡在 checkpoint（反复重启/等待提交）

症状：开机多次重启或长时间"正在启动"；logcat 样例：`KeyStorage: Committing upgraded keys` 迟迟不出现（KeyStorage.cpp:239-240 在等 `vold.checkpoint_committed=1`）；`KeyStorage: Wrote upgraded key to ... delaying commit due to checkpoint`（:366-367）。

定位命令：`vdc checkpoint supportsCheckpoint`、`vdc checkpoint needsCheckpoint`（vdc.cpp:135-155）；`getprop vold.checkpoint_committed`；`logcat -s Checkpoint vold KeyStorage`。

根因：checkpoint 未 commit：merger/update_engine 失败、/data 空间不足、或 boot retry 计数耗尽触发回滚（init 的 ExecVdcRebootOnFailure 语义，builtins.cpp:1228）。密钥侧的 blob 升级全部挂在 `keymaster_key_blob_upgraded` 未替换（KeyStorage.cpp:216-236）。

修复：系统面确认健康后 `vdc checkpoint commitChanges` 手动提交；需要回退时 `vdc checkpoint restoreCheckpoint`。回滚后 `keymaster_key_blob_upgraded` 自动作废、旧 blob 继续有效——密钥与系统面的原子性由此保证（第九章分析 5）。

场景五：vold 初始化 FBE 失败（开机进 recovery/反复重启）

症状：logcat 样例：`vdc: Command: cryptfs enablefilecrypto Failed: ...`（vdc.cpp:65 checkStatus 格式）；`FsCrypt: Failed to prepare user 0 storage`（FsCrypt.cpp:492）；`Vold unable to connect to keystore2.`（Keystore.cpp:111）。

定位命令：`getprop ro.crypto.type`（应为 file）；`vdc cryptfs mountFstab <blkdev> <slot>`（vdc.cpp:127）重放挂载；`vdc cryptfs enablefilecrypto` 与 `vdc cryptfs init_user0`（重放 init 两步）；`logcat -s vold FsCrypt MetadataCrypt Keystore`。

根因：a) fstab 的 fileencryption= 选项与内核能力不匹配（EncryptionOptions 解析失败，FsCrypt.cpp:239-294）；b) /data 上残留旧策略而新选项不一致——/data/unencrypted/mode 与 ref 的核对拒绝（第八章 8.1）；c) keystore2 未起/KeyMint HAL 缺失（容器包装依赖 TEE）。

修复：a) 核对 fstab 与 ro.crypto.volume.options；b) 策略变更设备必须 wipe /data（一致性校验不允许降级）；c) 先查 `dumpsys keystore2` 是否可达再重试。出厂产线首次启用顺序必须是 enablefilecrypto → init_user0 → 首次设锁，跳步会得到"有 DE 无 CE 绑定"的半成品状态。

存储与加密核心速查表

机制	一句话	源码锚点	所在册
FBE 双域	/data/user_de(DE开机即解) vs /data/user(CE解锁后解)	FsCrypt.cpp:842-873	本册
密钥链五层	PIN→Gatekeeper→SP→DE/CE Key→文件密钥	FsCrypt+LSS+KeyStorage	本册
Synthetic Password	CE key 由 SP 派生（改 PIN 不重加密）	LockSettingsService:202	本册
Gatekeeper	TEE 校验+防暴力（HIDL 1.0 冻结）	hardware/interfaces/gatekeeper	本册
keystore2	Rust 重写（service.rs:54 按安全级路由）	keystore2/src/service.rs	本册
KeyMint	TEE/StrongBox 硬件密钥（永不出硬件）	hardware/interfaces/keymint	本册
attestation	证书链（远程证明设备可信）	keystore2/attestation_key_utils	本册
direct boot	DE 解后闹钟可用，CE 解后 App 数据可见	AMS:5149	本册
vold Checkpoint	OTA 期间 /data 快照（写前重定向）	vold/Checkpoint.cpp	本册
FUSE passthrough	13：命中规则后内核直通（免 MediaProvider 中转）	vold/FsCrypt	本册